

RestroOS

A complete tester's guide to the three panels, every term explained, with step-by-step flows and expected results.

Build tested: commit `a466cff` · **Environment:** Production · **Test restaurant:** Chai Chakana Company (slug `ccc`), Kondapur branch

READ THIS BEFORE SHARING THE DOCUMENT

This handbook contains **live production passwords for a real business**. Anyone holding it can place real orders, change a real menu, and read real customer data. Do not email it broadly, upload it to a public tracker, or paste it into a group chat. Share it person-to-person, and have the passwords rotated once testing finishes.

Everything you do here is **real**. There is no separate sandbox. Orders you place appear in the owner's live order history and revenue figures.

Contents

- 1 What RestroOS is
- 2 Access & credentials
- 3 Glossary — every term
- 4 Panel A — Storefront (customer)
- 5 Panel B — Tenant Admin (restaurant)
- 6 Panel C — Platform Admin (RestroOS)
- 7 How money is calculated
- 8 How stock deducts automatically
- 9 End-to-end test scenarios
- 10 Known state & gotchas
- 11 How to report a bug

1 What RestroOS is

RestroOS is a **multi-tenant restaurant platform**. One deployment serves many independent restaurants at once, and each restaurant's data is walled off from every other. There are three separate web applications, each for a different kind of user.

PANEL	WHO USES IT	WHAT IT DOES
Storefront	The public / diners	Browse the menu, add to cart, apply a coupon, place and track an order. No login needed to browse; login required to order.
Tenant Admin	Restaurant owner & staff	Runs the restaurant: taking orders at the counter, the kitchen screen, menu, stock, staff, coupons, reports.
Platform Admin	RestroOS operators (you)	Creates and suspends restaurants on the platform. Sits above every tenant.

THE SINGLE MOST IMPORTANT CONCEPT

A **tenant** is one restaurant business. A **branch** is one physical outlet of it. Almost everything in the app is scoped to one or the other — the menu belongs to a branch, but ingredients and coupons belong to the whole tenant. When something behaves unexpectedly, the first question to ask is always: *which branch am I looking at?*

2 Access & credentials

Panel A — Storefront

FIELD	VALUE
URL	<code>https://storefront-liard-ten.vercel.app/cc</code>
Login	Register your own customer account, or use the tenant-admin account below if it exists as a customer. Browsing needs no login.
Note	The <code>/cc</code> on the end is the restaurant's slug. Drop it and you get a generic landing page, not the menu.

Panel B — Tenant Admin

This login has **three** fields, not two. The slug is what tells the system which restaurant you belong to.

FIELD	VALUE
URL	<code>https://tenant-admin-gules.vercel.app</code>
Restaurant Slug	<code>cc</code>
Email	<code>vaibhavnawale002@gmail.com</code>
Password	<code>Admin@123</code>
Role	Owner — full access, all branches, every screen

Panel C — Platform Admin

FIELD	VALUE
URL	<code>https://platform-admin-rho.vercel.app/tenants</code>
Account 1	<code>vaibhavnawale002@gmail.com</code> / <code>12345678</code>
Account 2	<code>avinashj387@gmail.com</code> / <code>1234</code>

TWO ACCOUNTS, UNVERIFIED WHICH IS WHICH

Both pairs above were supplied for the platform admin panel. If one fails, try the other before reporting a bug. Also note both passwords are weak (`1234` , `12345678`) — that is itself worth raising as a finding for a panel that controls every restaurant on the platform.

Backend API

Not a panel, but useful when a screen misbehaves and you need to know whether the fault is the interface or the server: `https://restroos-api.vercel.app/api/v1/health` should return `{"success":true}`.

3 Glossary — every term

These words appear throughout the interface. Understanding them is the difference between filing a real bug and filing a misunderstanding.

Structure

Tenant

One restaurant business on the platform. "Chai Chakana Company" is a tenant. Its data is invisible to every other tenant.

Slug

The short text ID for a tenant, used in URLs and at login. Here it is `ccc`. It is how the storefront knows whose menu to show.

Branch

One physical outlet. This tenant has one: **Kondapur**. Menu items and stock belong to a branch; ingredients, coupons and staff belong to the tenant as a whole.

Owner

A tenant admin with *no* branch assigned. Unrestricted across all their branches, and the only role that can manage staff, branches, and feature toggles.

Branch Admin

A tenant admin locked to a single branch. Sees only that branch's data, and only the screens an Owner has granted them.

Permission

A switch an Owner grants a Branch Admin, controlling one area (e.g. `manage_menu`). Owners bypass permissions entirely. See §5 for the full list.

Feature toggle

A tenant-wide off-switch, found under **Features**. Unlike a permission, it blocks *everyone* including the Owner. Useful to check when a screen is unexpectedly missing.

Menu

Category

A grouping on the menu — Tea, Snacks, Momos. Controls the section headings customers scroll through.

Menu Item

One sellable product with a name, price, image, and veg/non-veg marker.

Option Group / Add-on

Choices attached to an item that may change its price — e.g. Irani Chai offers **Size** and **Add-ons**. Options are always re-priced on the server when an order is placed, never trusted from the browser.

Veg indicator

The FSSAI marker: a green **dot** for vegetarian, a brown **triangle** for non-vegetarian. Shape matters, not just colour — this is a dietary requirement, so a colour-only marker would be a genuine bug.

Bestseller

An item flagged as popular. Shows an amber chip and surfaces in "Recommended".

Orders

Delivery Type

One of **Delivery**, **Dine In** (requires a table number), or **Takeaway**. This determines which status sequence the order follows.

Order Status

Where the order is in its life. Only ever moves forward, never backward. See §8.

KOT

Kitchen Order Ticket. The slip the kitchen cooks from — items and quantities only, no prices.

Bill

The customer-facing receipt, with prices, taxes and totals.

Walk-in Guest

A placeholder customer used for counter orders where staff collect no personal details. You will see many of these in the order list; that is normal.

Inventory

Ingredient

A raw material — Tea Leaves, Milk, Paneer. Shared across all branches of the tenant, but its *stock level* is tracked per branch.

Base Unit

The unit an ingredient is counted in: `g`, `kg`, `ml`, `l`, or `pc`. **Fixed forever once set** — changing it would corrupt every historical record, so the field is disabled when editing.

Unit type

Weight (g/kg), Volume (ml/l), or Count (pc). Conversion works within a type (kg→g) but is **rejected across types** — you cannot express millilitres in grams. A recipe attempting that should show a clear error, not silently miscalculate.

Recipe

What one serving of a menu item consumes. Garam Tea = 4g Tea Leaves + 120ml Milk + 8g Sugar + 2g Ginger + 0.3g Cardamom + 100ml Water. This is what makes stock deduct automatically.

Ledger

The stock history. It is **append-only**: nothing is ever edited or deleted. A correction is a new row that cancels the old one. Every row records the balance before and after, so any figure can be traced.

Transaction types

OPENING_STOCK — initial or restocked quantity (+).

CONSUMPTION — used by an order, automatic (−).

WASTAGE — spillage or spoilage, needs a reason (−).

ADJUSTMENT_IN / ADJUSTMENT_OUT — correction after a physical stock count (±).

REVERSAL — credits stock back when an order is cancelled (+).

Low stock

An ingredient at or below its threshold. Flagged on the Inventory dashboard.

Money**Coupon**

A discount code. May be a percentage or a fixed amount, and may carry a cap, a minimum order value, an expiry, and usage limits.

CGST / SGST

Central and State GST, **2.5% each** (5% total), applied *after* any discount. See §7.

4 Panel A — Storefront

The customer-facing shop at `/ccc`. Test this on a **real phone** as well as a desktop browser — it is designed mobile-first and some bugs only appear at narrow widths.

4.1 Screens

SCREEN	PURPOSE
Menu (home)	Browse by category, search, filter by Pure Veg / Non-Veg / Bestseller, add to cart
Cart	Review items, change quantities, remove lines
Checkout	Choose delivery type, address, payment method; apply coupon; place order
Orders	Past and live orders
Order detail	Live status tracker, itemised bill, print / email bill, reorder, cancel
Addresses	Saved delivery addresses
Profile	Account details

4.2 Register a customer account

- 1 Open `https://storefront-liard-ten.vercel.app/ccc`.
- 2 Tap **Log In** (top right), then **Register**.
- 3 Enter full name, email, phone, password. Use a real address you control if you want to test the email-bill feature.
Expected: Account created and you are signed straight in.

Accounts are **per tenant**. Registering on `/ccc` does not create an account at any other restaurant.

4.3 Order a customisable item and apply a coupon

This is the single most valuable storefront flow to test, because it exercises option pricing, discount capping and tax at once.

- 1 On the menu, find **Irani Chai** (Tea category, ₹20).
- 2 Tap **Add**. Because it has options, a customisation dialog opens rather than adding directly.
Expected: Dialog shows a **Size** group and an **Add-ons** group.
- 3 Choose a size and one or more add-ons. Watch the price update as you select.
Expected: The running price reflects each option's price change immediately.
- 4 Confirm. The Add button becomes a – 1 + stepper.
- 5 Raise the quantity to 4 or 5, so the subtotal comfortably exceeds ₹150. *This matters for the next step.*
- 6 Go to **Cart**, then **Checkout**.
- 7 Enter coupon code `CHAI20` and apply it.
Expected: 20% comes off — but never more than ₹30, however large the order.

- 8 Choose a payment method and place the order.

| **Expected:** Order confirmed, and it appears under Orders with status **Pending**.

THE POINT OF STEP 5 — TESTING THE CAP

CHAI20 is 20% off **capped at ₹30**. The cap only proves itself on an order above ₹150, because 20% of ₹150 is exactly ₹30. On a ₹100 order you would see ₹20 off and learn nothing. Deliberately build a ₹300 order: the discount must still be **₹30**, not ₹60. If it shows ₹60, that is a serious bug — file it immediately.

4.4 Coupons available for testing

CODE	DISCOUNT	CAP	PER-CUSTOMER LIMIT	EXPIRES
CHAI20	20%	₹30	None — reusable	Never
DOSA20	20%	None	Once only	22 Sep 2026

Test both, because they fail in different ways. Use `DOSA20` twice on the same account: the second attempt **must be rejected**. Use `CHAI20` repeatedly: it should keep working. Also try an invented code such as `NOTREAL99` — it should be refused with a clear message, not a crash.

4.5 Track, cancel and reorder

- 1 Open the order from **Orders**.
- 2 Check the status tracker. Completed stages show ticks; a delivered order must show a tick on **Delivered** — not a number.
- 3 While the order is still **Pending** or **Accepted**, a **Cancel Order** button is available.

| **Expected:** Cancelling works, and status becomes **Cancelled**.
- 4 Have staff advance the order to **Preparing** in Tenant Admin, then return here.

| **Expected:** The cancel button is now gone — the kitchen has started.
- 5 On any past order tap **Reorder**.

| **Expected:** The same items land in your cart.

5 Panel B — Tenant Admin

Sign in with slug `ccc`, `vaibhavnawale002@gmail.com`, `Admin@123`. You are an **Owner**, so every screen below is visible to you.

5.1 Every screen

SCREEN	WHAT IT IS FOR
Dashboard	Today at a glance — orders, revenue, live activity
Analytics	Trends over time: revenue, average order value, cost of goods, margin
Orders	Every order. Filter by status, date range and branch; see revenue for the filtered set; print KOT or bill
Customers	Everyone who has ordered
Take Order (POS)	The counter till — staff place orders for walk-in customers
Kitchen	Live cooking queue with timers
Tables	Dine-in floor plan and table setup
Menu	Items, prices, images, veg flags, option groups, and each item's recipe
Inventory	Ingredients, branch stock, the ledger, and the stock dashboard
Reports	Tax summary, payment breakdown, sales, category sales, menu profitability, stock valuation
Categories	Menu groupings and their display order
Coupons	Create and manage discount codes
Branches	Outlets (<i>Owner only</i>)
Staff	Admin accounts and their permissions (<i>Owner only</i>)
Integrations	Payments, notifications, realtime keys
Activity Log	Audit trail of staff actions
Settings / Features	Restaurant details, branding, tenant-wide feature toggles

5.2 Add a menu item — full walkthrough

- 1 Go to **Categories** first and confirm a suitable category exists. An item cannot be filed without one.
- 2 Go to **Menu** → **Add Item**.
- 3 Fill in name, price, category, and description.
- 4 Set the **veg / non-veg** flag correctly — this drives the FSSAI marker customers rely on.
- 5 Upload an image. Items without one show a placeholder, which looks unfinished on the storefront.
- 6 Optionally mark it **Bestseller** to surface it under Recommended.
- 7 Save, then open the storefront and confirm it appears in the right category.

Expected: Visible to customers within a refresh.

5.3 Add a coupon

- 1 **Coupons** → **Add Coupon**.
- 2 Enter a code. Keep it short and memorable — customers type this.
- 3 Choose **Percentage** or **Fixed amount**.
- 4 Set the value. For a percentage, **set a maximum discount** or a large order will hand away far more than intended.
- 5 Optionally set minimum order value, valid dates, total usage limit, and per-customer limit.
- 6 Save, then test it at storefront checkout.

Expected: Applies correctly, and respects every limit you set.

5.4 Set up inventory tracking for an item

Four steps, in this order. Skipping any one means stock will not deduct.

- 1 **Inventory** → **Ingredients** → **Add Ingredient**. Name, base unit, cost per unit, low-stock threshold. Remember the base unit is permanent.
- 2 **Inventory** → **Branch Stock** → **Add Opening Stock**. Without this the ingredient exists but has a balance of zero.
- 3 **Menu** → **pick the item** → **Recipe tab**. Add each ingredient and the quantity used per serving.
- 4 Place an order for that item and move it to **Preparing**.

Expected: Stock drops by exactly the recipe amount, and a CONSUMPTION row appears in Transactions.

5.5 Record wastage and correct a stock count

- 1 **Inventory** → **Branch Stock**, find the ingredient, choose **Wastage**. Enter quantity and a reason (required).
- 2 For a physical count, choose **Adjustment** and enter *what you actually counted* — not the difference.

Expected: The system works out the gap itself and writes ADJUSTMENT_IN or ADJUSTMENT_OUT.

5.6 Add a staff member and test permissions

This is the highest-value security test in the whole application.

- 1 **Staff** → **Add Admin**. Name, email, password.
- 2 Assign a **branch**. This makes them a Branch Admin rather than an Owner.
- 3 Grant only a couple of permissions — say Orders and Menu — and leave the rest unticked.
- 4 Sign out. Sign in as that new admin (same slug `ccc`).
- 5 Now try to reach a forbidden screen by typing its URL directly, e.g. `/coupons` or `/admins`.

Expected: Blocked. Hiding a menu link is not security — the server must refuse too. If the page loads, that is a critical bug.

Grantable permissions

PERMISSION	GRANTS ACCESS TO
manage_orders	Orders, Take Order (POS), Kitchen
manage_tables	Tables
manage_menu	Menu
manage_categories	Categories
manage_ingredients	Ingredients & recipes
manage_inventory	Stock and transactions
view_customers	Customers
view_analytics	Analytics
view_reports	Reports
manage_coupons	Coupons
manage_integrations	Integrations
manage_branding	Branding / settings
view_activity_log	Activity log

Staff management and branch management are never grantable. They stay Owner-only by design — otherwise a Branch Admin could promote themselves to Owner. If you find any way to reach them as a Branch Admin, that is a critical finding.

5.7 Take an order at the counter (POS)

- 1 **Take Order.** Pick a table for dine-in, or switch to Takeaway.
- 2 Search or browse, tap items to build the order, adjust quantities with the stepper.
- 3 Place the order.
 - | **Expected:** It appears in Orders as Pending and on the Kitchen screen.
- 4 Open **Kitchen** and advance it through its stages.
 - | **Expected:** The timer ticks live; the storefront tracker updates for the customer.

6 Panel C — Platform Admin

The operator panel, above all restaurants. Handle with care: actions here affect entire businesses.

6.1 What to test

- 1 Sign in at `https://platform-admin-rho.vercel.app`.
- 2 Open **Tenants** to list every restaurant on the platform.
- 3 Create a **new test tenant** — give it an obvious throwaway name and slug such as `qa-test-01`.
| **Expected:** Tenant created with its own owner account and its own storefront at `/qa-test-01`.
- 4 Visit that new storefront URL.
| **Expected:** An empty menu — **not** Chai Chakana's items. Any leakage between tenants is a critical bug.
- 5 Suspend the test tenant, then try to sign into its admin panel.
| **Expected:** Login refused, and any already-signed-in staff lose access on their next action.

NEVER SUSPEND CHAI CHAKANA COMPANY

It is a live business. Suspending it takes a real restaurant offline. Only ever suspend tenants you created yourself for testing.

7 How money is calculated

Check totals by hand at least once. The order is important — **tax is applied after the discount**, not before.

```
Subtotal = sum of (item price + option price changes) × quantity
Discount = coupon applied to subtotal, respecting its cap
Taxable amount = Subtotal - Discount
CGST = Taxable × 2.5%
SGST = Taxable × 2.5%
Total = Taxable + CGST + SGST
```

Worked example

Five Irani Chai at ₹20, plus ₹10 of add-ons each (₹30 per cup) = **₹150.00** subtotal. `CHAI20` gives 20% = ₹30, exactly at the cap, so discount is **₹30.00**. Taxable = ₹120.00. CGST ₹3.00 + SGST ₹3.00. **Total ₹126.00**.

Now double it — ten cups, ₹300 subtotal. 20% would be ₹60, but the cap holds it at **₹30**. Taxable ₹270, CGST ₹6.75, SGST ₹6.75, **total ₹283.50**. If you see a ₹60 discount here, the cap is broken.

8 How stock deducts automatically

Status sequences

Delivery: Pending → Accepted → Preparing → Ready → Out For Delivery → Delivered

Dine In / Takeaway: Pending → Accepted → Preparing → Ready → Delivered

Cancelled: reachable from Pending / Accepted / Preparing (staff), or Pending / Accepted (customer)

Status only ever moves **forward**. An attempt to move backward must be refused.

The trigger

Stock deducts at exactly one moment: **when an order enters "Preparing"** — the point the kitchen actually starts cooking. Not when the order is placed, not when it is delivered.

- Deduction is **idempotent** — moving through Preparing twice never double-deducts.
- Cancelling a Preparing order writes **REVERSAL** rows crediting every ingredient back exactly.
- An item with **no recipe** deducts nothing. This is expected, not a bug.
- Stock is allowed to go **negative**. This is deliberate: the kitchen must not be blocked mid-service by a paperwork error. A negative balance is a signal to do a stock count.

Verifying it

- 1 **Inventory → Branch Stock**. Write down the balance of an ingredient — say Tea Leaves.
- 2 Place an order for a mapped item (e.g. Garam Tea) and move it to **Preparing**.
- 3 Return to Branch Stock.
 - Expected: Tea Leaves has fallen by exactly 4g per serving.
- 4 **Inventory → Transactions**, filter by that ingredient.
 - Expected: A CONSUMPTION row referencing the order number, showing the before → after balance.
- 5 Now cancel that order and look again.
 - Expected: A REVERSAL row, and the balance back exactly where it started.

9 End-to-end test scenarios

Each scenario spans more than one panel — which is where the real bugs live.

E2E-1 • Full customer journey

- 1 Storefront: register, add Irani Chai with options, apply `CHAI20`, place a Dine In order.
- 2 Tenant Admin → Orders: the order is listed as Pending, with the correct discounted total.
- 3 Kitchen: advance to Preparing.
- 4 Inventory → Transactions: CONSUMPTION rows written against that order.
- 5 Storefront → order detail: the tracker has moved.
- 6 Kitchen: advance to Delivered.
| **Expected:** Tracker shows a tick on Delivered — not a number — and the order leaves the kitchen queue.

E2E-2 • Cancellation restores stock

- 1 Note the balance of every ingredient in one item's recipe.
- 2 Order it, advance to Preparing, confirm the balances drop.
- 3 Cancel the order from Tenant Admin.
| **Expected:** Every balance returns to its exact starting figure, via REVERSAL rows. No row is ever edited or deleted.

E2E-3 • Tenant isolation (critical)

- 1 Create `qa-test-01` in Platform Admin.
- 2 Open its storefront.
| **Expected:** Empty menu. None of Chai Chakana's 45 items.
- 3 Sign into its admin panel; check Orders, Customers, Inventory.
| **Expected:** All empty. Any Chai Chakana data appearing here is a critical breach.
- 4 While signed in as the `ccc` Owner, try opening an order belonging to the other tenant by guessing its ID in the URL.
| **Expected:** Not found. Never another tenant's data.

E2E-4 • Permission enforcement (critical)

Follow §5.6, then attempt to reach every ungranted screen by direct URL. The sidebar hiding a link proves nothing; the server must refuse.

E2E-5 • Coupon limits

- 1 Apply `CHAI20` to a ₹300 order.
| **Expected:** Discount is ₹30, not ₹60.
- 2 Use `DOSA20` once, then attempt it again on the same account.
| **Expected:** Second attempt refused — it is limited to one per customer.

3 Try NOTREAL99 .

Expected: A clear "invalid coupon" message, not a crash or a silent zero discount.

E2E-6 • Mobile layout sweep

On a real phone, walk every storefront screen and watch for text escaping its button, content hidden behind the bottom bar, or sideways scrolling. Repeat in landscape.

10 Known state & gotchas

YOU WILL SEE	WHY
Orders from "Walk-in Guest"	Normal — counter orders with no customer details collected.
Customers named "Verify Tomato Soup", "E2E ...", "Inventory Demo Customer"	Test orders (#228, #229, #230, #237–#240) created while verifying inventory. Not bugs. Some sit in Preparing; #230 is Cancelled.
Ingredient spelled " Refinded Oil "	A real typo in live data, referenced by 12 recipes. Known — no need to report.
Cardamom flagged low stock	Deliberately left at 15g against a 20g threshold to demonstrate the alert.
Most menu items have no photo	Content gap, not a fault.
Revenue looks inflated	The test orders above are included in totals.
Stock going negative	Deliberate. Service is never blocked by a paperwork error.

RATE LIMITING

The API allows 120 requests a minute, and only 20 attempts per 15 minutes on login and registration. Rapid repeated testing will start returning "*Too many attempts*". That is the protection working correctly — wait a few minutes rather than filing a bug.

11 How to report a bug

A report that cannot be reproduced cannot be fixed. Include all of the following:

FIELD	EXAMPLE
Panel & screen	Storefront → Checkout
Account used	Customer <code>tester1@example.com</code> / Owner on <code>ccc</code>
Device & browser	iPhone 14, Safari 17 — or Windows 11, Chrome 128
Exact steps	Numbered, from a signed-out state, so anyone can follow them
Expected	Discount capped at ₹30
Actual	Discount showed ₹60
Evidence	Screenshot or screen recording. For a phone, include the whole screen.

Severity

- **Critical** — one tenant can see another's data; a permission can be bypassed; money is calculated wrongly; orders or stock are lost.
- **High** — a core flow is blocked, with no workaround.
- **Medium** — a feature misbehaves but can be worked around.
- **Low** — cosmetic, wording, or alignment.

Anything touching **tenant isolation**, **permissions**, or **money** should be reported the moment you see it, without waiting to finish the round.

RestroOS QA Handbook · Build `a466c ff` · Test tenant: Chai Chakana Company (`ccc`), Kondapur branch. Contains live production credentials — do not distribute publicly, and rotate the passwords once testing is complete.